In Class Lab 5g

Procedures That Change Data | CIS 4720 Database Management and SQL

Points	Time	Work in	Turn in
10	About 90 minutes	lab5g_starter.sql	Your GitHub repository URL

Textbook: Coronel and Morris, *Database Systems: Design, Implementation, and Management*, 14th edition, Chapter 5, Section 5.8.
The due date is listed on Canvas LMS.

What this lab adds to Lab 5f

Every procedure you wrote in Lab 5f held one SELECT. It read data and showed it to you. That is useful, but the bigger reason databases have procedures is the other kind of job: **changing** data, the same way, correctly, every time.

In Lab 5e Step 9 you added a score for Jenna and then had to remember to recalculate her gpa by hand. Two statements, and the second one is easy to forget. Today you put both inside one procedure, so nobody can do the first without the second.

New in this lab	Where
A procedure with more than one parameter, filled in by position	Steps 3 and 4
A procedure body holding two statements, run in order	Step 3
INSERT and DELETE inside a procedure	Steps 3, 5 and 7
A procedure and a trigger working together on the same row	Steps 6 and 8
Two weaknesses in what you built, left for Lab 5h to fix	Step 9

Part 1. Get set up

Step 1. Connect to information_schema and make your folder

Make a folder on OneDrive called **CIS4720_Lab5g**, download **lab5g_starter.sql** from Canvas into it, and open the folder in VS Code. Connect SQLTools to **information_schema** as root, exactly as in Lab 5f.

Step 2. Create the schema and run the starter file

Run the three setup statements at the top of the file one at a time: **show databases;**, **create database if not exists cis_students_db;** and **use cis_students_db;**. Then add a second connection with Database set to **cis_students_db** and work from that one.

Run the rest of the file one statement at a time. It rebuilds everything from Lab 5e plus the three procedures you finished in Lab 5f. Each function, trigger and procedure is one statement from CREATE down to END, so select the whole block and run the selection.

Check	What you should see
CALL list_students();	Five students, highest gpa first: 92.33, 85.00, 84.00, 82.50, 0.00

Check	What you should see
SELECT * FROM attendance;	Four rows, ids 500 to 503

If you already have cis_students_db from Lab 5f

Every PC is wiped each night, so normally you will not. If you are on your own machine and the starter stops with **ERROR 1050, table already exists**, run **drop database cis_students_db;** while connected to information_schema, then start Step 2 again.

Part 2. A procedure that adds a score

Step 3. Create add_score

This procedure takes three values in and does two jobs: it stores the score, then brings the student's gpa up to date. The second job is written for you, so you can see the shape of a two statement body. You write the first.

```
-- adds one test score and keeps that student's gpa correct
CREATE PROCEDURE add_score(IN p_test_name VARCHAR(20),
                           IN p_student_id INT,
                           IN p_score FLOAT)
BEGIN
    -- job one: store the new score
    -- >>>>>  YOUR STATEMENT GOES HERE  <<<<<

    -- job two: recalculate that one student's gpa
    UPDATE students
    SET gpa = calculate_gpa(p_student_id)
    WHERE student_id = p_student_id;
END
```

Writing the statement that goes in the gap

An INSERT into test_scores, with the three parameters as the values.

1. INSERT INTO test_scores, then in brackets the three columns you are filling: test_name, student_id and score. Name them. test_score_id is a surrogate key and the server fills it in, so leave it out.

2. On the next line **VALUES**, then in brackets the three parameters in the same order as the columns: p_test_name, p_student_id, p_score.

3. No quotes around the parameters, even p_test_name. Quotes would store the words p_test_name rather than the test name that was passed in.

4. End it with a semicolon. Job two starts on the line after.

The order of the statements matters. The UPDATE runs second, so when calculate_gpa averages the scores, the new one is already there to be counted. Swap them and every gpa would be one score behind.

Step 4. Use add_score, and watch the gpa change

Values in a CALL are matched to parameters by **position**: the first value goes to the first parameter, and so on. For add_score that means test name, then student, then score. You write both calls:

Call add_score for	Test	Student	Score
Rita Nunez	quiz4	5	88
Jenna Wilson	quiz4	1	100

Then call list_students. Two gpas moved, and you never wrote an UPDATE:

first_name	gpa before	gpa after	Why
Maria Elena	92.33	92.33	Not touched
Jenna	85.00	90.00	Her 100 joined 90 and 80
Rita	0.00	88.00	Her first score
Robert	84.00	84.00	Not touched
Inder	82.50	82.50	Not touched

> **Order of the values is not checked for you**
>
> **CALL add_score(5, 'quiz4', 88)** puts 5 into p_test_name and quiz4 into p_student_id. The server cannot turn quiz4 into a number, so you get an error. Worse cases exist where both values fit and the wrong data is stored silently. Always check the parameter list before you write a CALL.

Part 3. A procedure and a trigger working together

Step 5. Write record_attendance

Most of this one is yours. There is no skeleton, only the specification and a walkthrough.

Requirement	Detail
Name	record_attendance
Parameters	Two IN parameters: p_student_id, INT, and p_status, VARCHAR(10)
Body	One INSERT into attendance that fills in student_id and status, and nothing else

Walking through it

1. Start from your Step 3 procedure as a model: CREATE PROCEDURE, the name, the parameter list in brackets with IN, a name and a type for each, separated by a comma.

2. BEGIN on its own line.

3. The INSERT. Name only the two columns student_id and status. Do not mention attendance_id or attended_on at all.

4. END on its own line. Select the whole block and run it.

Why leave attended_on out? Because the BEFORE INSERT trigger from Lab 5e fills in today's date when none is given. The procedure does not need to know that. It just inserts, and the trigger tidies the row on its way in.

Step 6. Test record_attendance

Record student **2** as **absent**, written in lower case, then select the attendance table. You write both statements.

attendance_id	student_id	attended_on	status	Who did it
504	2	today's date	ABSENT	The id came from AUTO_INCREMENT, the date and the capitals from the trigger

One CALL, and three different parts of the database each did a job: the procedure chose the row, AUTO_INCREMENT numbered it, and the trigger cleaned it up. That division of work is how real systems are put together.

Step 7. Create remove_score

The reverse of add_score. The DELETE is yours to write.

```
-- removes one named test score for one student
CREATE PROCEDURE remove_score(IN p_student_id INT,
                              IN p_test_name VARCHAR(20))
BEGIN
    -- delete only the row that matches BOTH values
    -- >>>>>  YOUR STATEMENT GOES HERE  <<<<<
END
```

Writing the statement that goes in the gap

1. DELETE FROM test_scores.

2. A WHERE clause with **two** conditions joined by **AND**: the student_id column equals p_student_id, and the test_name column equals p_test_name.

3. A semicolon, then leave END as it is.

Think about what each condition protects. Without the student_id test, removing Jenna's quiz4 removes everybody's quiz4. Without the test_name test, it removes every score Jenna has. Without a WHERE at all, it empties the table. A DELETE inside a procedure runs every time the procedure is called, so this is the line to read twice.

Notice there is no UPDATE in this one. You will see why in Step 8.

Step 8. Test remove_score

Remove Jenna's quiz4 score. She is student 1. Then look at her gpa. You write both statements.

Jenna's gpa	When
90.00	Before, with her quiz4 of 100 counted
85.00	After. Back to the average of 90 and 80

remove_score has no UPDATE in it, yet the gpa was corrected. The AFTER DELETE trigger you wrote in Lab 5e fired when the procedure's DELETE ran. Procedures and triggers do not need to know about each other to work together.

Part 4. What these procedures cannot do yet

Step 9. Two problems, left open on purpose

Your procedures do exactly what they are told, including when what they are told is wrong. Run these and, under each one, write a comment saying what happened and what **should** have happened. You write all three.

1. Add a **quiz5** score of **minus 20** for Inder, student 4. Then call list_students.
2. Remove that quiz5 score with remove_score, and call list_students again.
3. Record attendance for student **9** as present.

Call	What happens	The problem
add_score with a score of minus 20	Accepted. Inder's gpa drops from 82.50 to 48.33	A score below 0 should never get in. Nothing checked it
remove_score for that quiz5	Inder is back to 82.50	This is how you clean up
record_attendance for student 9	ERROR 1452, cannot add or update a child row, a foreign key constraint fails	The data is safe, but the message is written for a database administrator, not for the person who typed the wrong number

> **Where this goes next**
>
> Both problems have the same cure: the procedure should **check before it acts**, and refuse with a clear message of its own when something is wrong. That needs a few more pieces of procedural SQL, and they are what Lab 5h is about.

Step 10. List every routine in your schema

Write the query from Lab 5f Step 9 yourself this time, from memory or from your Lab 5f file: routine_name and routine_type from information_schema.routines, for cis_students_db only, in name order. You should get seven rows:

routine_name	routine_type	From
add_score	PROCEDURE	Today
calculate_gpa	FUNCTION	Lab 5d

routine_name	routine_type	From
list_students	PROCEDURE	Lab 5f
record_attendance	PROCEDURE	Today
remove_score	PROCEDURE	Today
scores_for_test	PROCEDURE	Lab 5f
student_report	PROCEDURE	Lab 5f

Part 5. Push to GitHub

Step 11. Publish to a new repository

Save lab5g_starter.sql first. If you are not signed in to GitHub, click **Accounts** at the bottom of the left bar, choose **Sign in with GitHub**, and click **Authorize**.

1. Click **Source Control**, then **Publish to GitHub**.
2. Name it **cis4720-lab5g**. It has to be a name you have not used.
3. Choose **Publish to a public repository**. A private one gives your instructor a 404 page instead of your work.
4. Tick **lab5g_starter.sql**. Leave the **.vscode** folder unticked, because that is the one holding your password.

> **Sign out of GitHub before you leave the lab**
>
> Shared PC. Your sign in stays on it until somebody removes it. Click **Accounts**, then your username, then **Sign Out**. Everything else on the PC is wiped tonight, so the repository is the only copy of your work.

Step 12. Check it and submit the URL

1. Open the repository on github.com, click the file and read it on the page. What is there is exactly what gets graded.
2. Open the URL in a private or incognito window. A **404** means it is private, so go to **Settings** and change the visibility to public.
3. Paste the URL into the Lab 5g submission box on Canvas.

Submit the URL even if you did not finish. Partial work your instructor can read scores better than finished work nobody can reach.

How it is graded

Steps	What is being checked	Points
1 to 2	Schema created and the starter file running	1.0
3	add_score created, with the INSERT you wrote	1.5
4	add_score called for Rita and Jenna	0.5
5	record_attendance written from the specification	2.0
6	record_attendance tested, row 504 shown	0.5
7	remove_score created, with a DELETE limited by both parameters	1.5
8	remove_score tested, Jenna back to 85.00	0.5
9	All three calls run, each with a comment on what should have happened	1.0
10	The routines query written and run	0.5
All	Required. A comment in your own words above every statement	1.0
11 to 12	A new public repository with the file in it, URL submitted	0.0

Total 10 points. The due date is listed on Canvas LMS.

If something goes wrong

What you see	What it means
ERROR 1064 near 'DELIMITER $$'	Delete the DELIMITER lines and the $$ after END.
ERROR 1064 near '--something'	A comment lost the space after its two dashes.
ERROR 1064 near the end of the body	Only part of the block ran. Select from CREATE PROCEDURE to END and run the selection.
ERROR 1304, PROCEDURE already exists	Run DROP PROCEDURE IF EXISTS and the name, then create it again.
ERROR 1318, incorrect number of arguments	The CALL has a different number of values than the procedure has parameters. add_score takes three.
ERROR 1366, incorrect integer value	The values in the CALL are in the wrong order. Check the parameter list.
ERROR 1452, cannot add or update a child row	A student_id that is not in students. Expected in Step 9, nowhere else.
The gpa did not change after add_score	The UPDATE is missing, or it comes before the INSERT. Drop the procedure and create it again in the right order.
The score was stored as the text p_test_name	Quotes around a parameter in VALUES. Remove them.